Resumo: Projeto e Análise de Algoritmos - Prova 2

Sumário

- Problema da Correspondência Estável
  - Escalonamento de Intervalos (Interval Scheduling)
  - Escalonamento de Intervalos Ponderado (Weighted Interval Scheduling)
- Algoritmos Gulosos (Greedy)
  - Exemplo Problema de Escalonamento de Intervalos
    - Regras gulosas possíveis (e seus problemas)
- Dividir e Conquistar (Divide and Conquer)
- Programação Dinâmica (PD)
  - Propriedades Fundamentais
  - Exemplo Problema de Escalonamento de Intervalos
    - Estrutura da solução via PD
  - Como Projetar de Algoritmos com PD
- Problema do Troco (Coin Change Problem)

Problema da Correspondência Estável

Escalonamento de Intervalos (Interval Scheduling)

Considere o seguinte problema de escalonamento: você possui **um único recurso** e várias pessoas desejam utilizá-lo por **intervalos de tempo específicos**.

Cada requisição (*request*) pode ser representada como:

> "Posso reservar o recurso a partir do instante (s) até o instante (f)?"

Assuma que o recurso pode ser utilizado por **no máximo uma pessoa de cada vez**. O objetivo é **selecionar o maior número possível de solicitações compatíveis**, ou seja, que **não se sobreponham no tempo**.

Exemplo ilustrativo

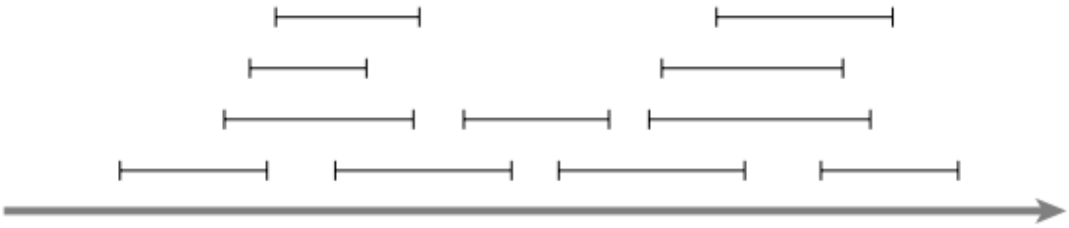


Figure 1.4 An instance of the Interval Scheduling Problem.

Nesse cenário, precisamos escolher o maior subconjunto de intervalos que **não se intersectam**.

Esse problema pode ser resolvido por um algoritmo **guloso (*greedy*)** bastante natural: primeiro, **ordenamos as solicitações segundo uma determinada heurística** (por exemplo, pela hora de término), e depois **as processamos de forma sequencial**, sempre escolhendo a próxima solicitação que **não conflita com as já aceitas**.

Essa abordagem representa o comportamento típico de um **algoritmo guloso**, ou seja, um método que toma **decisões locais**, passo a passo, sem planejar todo o futuro, mas que (em alguns casos) ainda produz **soluções ótimas**.

Escalonamento de Intervalos Ponderado (Weighted Interval Scheduling)

No problema clássico de Escalonamento de Intervalos, buscamos **maximizar o número de solicitações aceitas**. Agora, vamos considerar uma versão mais geral: **cada solicitação (i)** tem um **valor (ou peso)** associado, denotado por ($v_i > 0$).

Esse valor pode representar, por exemplo, **o lucro obtido** ao aceitar a solicitação (i). O objetivo passa a ser **maximizar o valor total das solicitações aceitas**, mantendo-as **compatíveis** (sem sobreposição).

Se todos os valores forem iguais a 1, ou seja, ($v_i = 1 \; \forall \; i$), então o problema se reduz ao caso básico do **Interval Scheduling**. Porém, quando os valores são arbitrários, o problema muda completamente de natureza.

Por exemplo:

> Se uma única solicitação (v_1) tiver um valor maior que a soma de todas as outras, então a solução ótima **deve incluir** essa solicitação, **independentemente** da configuração dos intervalos.

Portanto, **qualquer algoritmo para essa versão ponderada precisa levar os valores em consideração**, e ainda assim **se comportar corretamente como o caso não ponderado**

quando todos os (v_i) forem iguais.

Infelizmente, **nenhuma regra gulosa simples** é suficiente para resolver o problema ponderado de forma ótima. Em vez disso, empregamos uma técnica mais poderosa: a **Programação Dinâmica (PD)**.

Essa abordagem **constrói soluções ótimas parciais** de forma **incremental e tabular**, permitindo **combinar resultados intermediários** de maneira eficiente, até chegar à solução ótima final.

Algoritmos Gulosos (Greedy)

Os **algoritmos gulosos** (ou *greedy algorithms*) são uma classe de estratégias que constroem soluções **passo a passo**, sempre escolhendo, em cada etapa, a **melhor opção aparente naquele momento**, sem voltar atrás nem reconsiderar decisões anteriores.

Em outras palavras, um algoritmo guloso age de forma **míope**, buscando **otimizar localmente** algum critério de decisão (como tempo, custo ou lucro), na esperança de que essa sequência de escolhas locais leve a uma **solução globalmente ótima**.

Muitas vezes, é possível formular **várias versões gulosas** para o mesmo problema, dependendo de qual aspecto se tenta otimizar a cada passo. O interessante é que, quando **um algoritmo guloso realmente encontra a solução ótima**, isso revela algo **profundo sobre a estrutura do problema**, ou seja, existe uma **regra de decisão local** que, repetida corretamente, leva ao melhor resultado global.

Por outro lado, há muitos problemas em que as escolhas locais “aparentemente boas” **não conduzem à solução ótima**, e o desafio está justamente em **identificar quando a abordagem gulosa funciona** e **provar sua correção**.

Exemplo: Problema de Escalonamento de Intervalos

O Problema de Escalonamento de Intervalos é um ótimo exemplo para discutir **como funcionam os algoritmos gulosos**.

A ideia fundamental é adotar **uma regra simples** para escolher qual requisição aceitar primeiro. Uma vez escolhida uma requisição (i_1):

- todas as requisições **incompatíveis com (i_1)** são rejeitadas;
- selecionamos a próxima requisição (i_2) que satisfaça a mesma regra;
- e repetimos esse processo até esgotar todas as requisições.

O desafio é

Qual regra de escolha (heurística gulosa) produz a **melhor solução possível**?

Regras gulosas possíveis (e seus problemas)

1. Escolher a requisição que começa mais cedo

A regra mais intuitiva é **aceitar sempre a requisição disponível com o menor tempo de início**, ou seja, o menor ($s(i)$). Isso parece bom, pois o recurso começa a ser usado rapidamente.

Porém, essa estratégia **não garante uma solução ótima**. Se a primeira requisição escolhida ocupar um intervalo muito longo, ela **bloqueia o recurso** e **impede o agendamento de muitas outras requisições menores**.

Em casos extremos, por exemplo, se essa requisição termina no tempo máximo ($f(i)$) entre todas, o algoritmo aceitará apenas **uma requisição**, enquanto a **solução ótima** poderia aceitar **várias**.

2. Escolher a requisição com menor duração

Outra ideia é escolher a requisição que dura menos tempo, isto é, com menor intervalo:

$$f(i) - s(i)$$

Essa regra tende a liberar o recurso mais rápido e, em muitos casos, é melhor que a anterior.

No entanto, **ainda pode gerar soluções subótimas**. Por exemplo, imagine três intervalos onde o do meio é curto, mas **conflita com os outros dois**. Aceitar o curto (localmente melhor) impede aceitar os outros dois (globalmente melhor).

3. Escolher a requisição com menos conflitos

Uma terceira heurística considera o **número de incompatibilidades** de cada requisição. Selecionamos aquela que conflita com **o menor número de outras requisições**.

Essa escolha pode resolver o problema do exemplo anterior, já que o intervalo central (que causava conflito com dois outros) deixaria de ser escolhido.

Contudo, essa regra também **não é perfeita**. Há configurações onde o algoritmo aceita um conjunto **subótimo**, deixando de alcançar o máximo possível de intervalos compatíveis.

4. Escolher a requisição que termina mais cedo

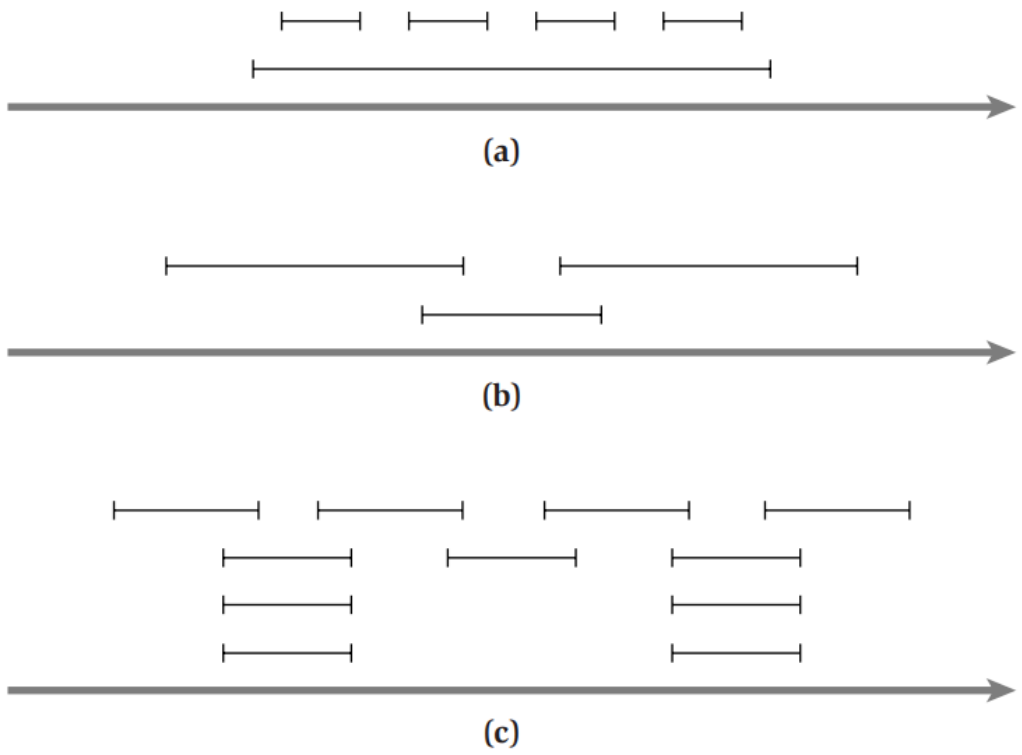
Por fim, temos a regra **correta** para o Problema de Escalonamento de Intervalos:

> Escolher sempre a requisição que **termina mais cedo**, isto é, com o menor tempo de término $f(i)$.

A ideia é simples: ao escolher quem termina antes, o recurso fica **livre o quanto antes**, permitindo acomodar **o maior número possível de requisições futuras**.

Essa regra evita o erro das anteriores — ela **não tenta maximizar o uso imediato**, nem **minimizar a duração**, nem **contar conflitos**. Ela apenas busca **maximizar a flexibilidade** para as próximas escolhas.

Exemplo ilustrativo



- Algumas instâncias do Problema de Escalonamento de Intervalos em que algoritmos gananciosos naturais não conseguem encontrar a solução ótima.
  - Em (a), não funciona selecionar o intervalo que começa mais cedo;
  - Em (b), não funciona selecionar o intervalo mais curto;
  - Em (c), não funciona selecionar o intervalo com o menor número de conflitos.

Conclusão intuitiva

Esses exemplos ilustram um ponto crucial sobre **algoritmos gulosos**:

> Uma escolha **ótima localmente** — aquela que parece a melhor no momento — nem sempre leva a uma **solução ótima globalmente**.

A dificuldade em projetar algoritmos gulosos está em encontrar **a regra de escolha correta**, isto é, aquela que garante que o **ótimo local leva ao ótimo global**.

No caso do Problema de Escalonamento de Intervalos, essa regra existe e é **selecionar sempre a requisição que termina mais cedo**. Esse método **gera uma solução ótima global**.

Dividir e Conquistar (Divide and Conquer)

Dividir para conquistar refere-se a uma classe de técnicas algorítmicas nas quais se divide a entrada em várias partes, resolve-se o problema em cada parte recursivamente e, em seguida, combinam-se as soluções para esses subproblemas em uma solução geral. Em muitos casos, pode ser um método simples e poderoso.

Programação Dinâmica (PD)

Enquanto os **algoritmos gulosos** tomam **decisões locais imediatas**, na esperança de que levem ao melhor resultado global, a **programação dinâmica (PD)** segue uma estratégia **planejada e sistemática**.

Ela **explora implicitamente todas as soluções possíveis**, mas o faz de forma **organizada, eficiente e com reuso de resultados intermediários**.

A essência da PD é:

1. **Decompor o problema** em vários **subproblemas menores**;
2. **Resolver e armazenar** as soluções desses subproblemas (evitando recalcular);
3. **Combinar** as soluções parciais para formar a resposta ao problema original.

Dessa forma, a programação dinâmica **elimina redundâncias** típicas da força bruta, que reavalia as mesmas situações várias vezes, e transforma problemas de **complexidade exponencial** em **algoritmos polinomiais**.

Pode-se dizer que a programação dinâmica **atua no limite da força bruta**, pois considera o espaço completo de soluções, porém de forma **inteligente e estruturada**, **reutilizando resultados já computados** por meio de **armazenamento intermediário**.

Propriedades Fundamentais

- **Subestrutura ótima**
  A solução ótima de um problema pode ser construída a partir das soluções ótimas de seus subproblemas.
- **Subproblemas sobrepostos**
  Os mesmos subproblemas surgem repetidamente e devem ser resolvidos **apenas uma vez**, armazenando seus resultados.

Tipos de Abordagem

- Top-Down (recursiva com *memoization*)
  A solução começa no problema principal e **chama recursivamente** os subproblemas. Cada resultado calculado é **armazenado em cache** (geralmente em um vetor ou mapa) para **evitar recomputações**.
  - A **memoization** é um **armazenamento sob demanda**: o subproblema só é resolvido e guardado **quando necessário**.
  - Essa abordagem é **conceitualmente simples**, pois segue diretamente a definição recursiva do problema.
- Bottom-Up (iterativa com tabulação)
  Os subproblemas são resolvidos **do menor para o maior**, **preenchendo uma tabela iterativamente** até chegar à solução global.
  - A **tabulação** é um **armazenamento proativo**: todos os subproblemas menores são resolvidos **em ordem crescente**, antes de usá-los.
  - A tabela atua como uma **forma explícita de memoization**, pois cumpre o mesmo papel de reter resultados intermediários, mas **sem recursão**.
  - Essa abordagem é geralmente **mais eficiente em tempo e espaço**, evitando overheads de chamadas recursivas.

Exemplo: Problema de Escalonamento de Intervalos Ponderados

No **Problema de Escalonamento de Intervalos Ponderados**, cada requisição i possui um **valor ou peso** $v_i > 0$. O objetivo é **maximizar o valor total** das requisições compatíveis, e não apenas o número delas.

> **Observação**
>
> O caso em que $v_i = 1$ para todo i corresponde ao problema anterior (sem pesos), resolvido de forma ótima por um algoritmo guloso. Porém, ao introduzir pesos diferentes,

a natureza do problema muda completamente: **nenhuma heurística gulosa simples** é capaz de garantir a solução ótima.

É nesse ponto que a **programação dinâmica** se torna a ferramenta ideal, pois consegue **explorar todas as combinações possíveis de intervalos compatíveis**, de maneira **eficiente e estruturada**.

Estrutura da solução via PD

Construímos um **vetor de soluções parciais** M, onde cada entrada $M[j]$ representa o **valor máximo possível** considerando **apenas os** j **primeiros intervalos** (ordenados por tempo de término).

A recorrência fundamental é:

$$M[j] = \max\Big(v_j + M[p(j)],\ M[j-1]\Big)$$

onde:

- v_j é o **peso** do intervalo j;
- $p(j)$ é o **índice do último intervalo compatível** com j (isto é, o maior $i < j$ tal que $f_i \leq s_j$).

Como Projetar de Algoritmos com PD

Para desenvolver um algoritmo baseado em programação dinâmica, é necessário definir um conjunto adequado de subproblemas derivados do problema original. Esses subproblemas devem satisfazer algumas propriedades fundamentais:

1. **Definir o conjunto de subproblemas:**
   Escolher uma decomposição do problema original que produza um número **polinomial** de subproblemas distintos.
2. **Estabelecer a relação de recorrência:**
   Identificar **como subproblemas menores** se combinam para formar soluções maiores.
3. **Escolher a ordem de resolução:**
   Determinar uma sequência **natural e coerente** de resolução (recursiva ou iterativa).
4. **Decidir a estratégia de armazenamento:**
   - **Top-Down:** armazenar conforme o cálculo é feito (memoization).
   - **Bottom-Up:** preencher a tabela em ordem crescente (tabulação).

Esses passos são **princípios de projeto**, e não regras fixas. A escolha entre top-down e bottom-up depende da **recorrência** e da **estrutura natural** do problema.

Resumo Geral

Técnicas de Design de Algoritmos

- **Guloso (Greedy)**
  Constrói uma solução incrementalmente, tomando em cada passo a decisão que parece melhor localmente com base em algum critério.
  Estratégias de prova/justificação comum:
  - **Greedy stays ahead (o guloso mantém vantagem):** mostre que, após cada passo, a solução do algoritmo guloso é pelo menos tão boa quanto a de qualquer outra solução comparável.
  - **Limite estrutural (structural bound):** identifique uma propriedade que todas as soluções válidas devem satisfazer e prove que a solução gulosa atinge essa propriedade otimamente.
  - **Argumento de troca (exchange argument):** transforme uma solução ótima por meio de trocas sucessivas até obter a solução do algoritmo guloso, sem degradar a qualidade.
- **Dividir e Conquistar (Divide and Conquer)**
  Divida o problema em subproblemas **independentes**, resolva cada um recursivamente e combine as soluções parciais para formar a solução global. Muito usado quando subproblemas são disjuntos e fáceis de recombinar.
- **Programação Dinâmica (PD)**
  Decompõe o problema em subproblemas **sobrepostos**, resolve e armazena soluções parciais e, em seguida, combina essas soluções para construir a solução do problema maior. Ideal quando existe **subestrutura ótima** e **subproblemas que se repetem**.

Prós e Contras (visão prática)

Objetivo comum: projetar algoritmos eficientes (geralmente em tempo polinomial).

1. **Guloso**
   - **Prós:** abordagem natural para o projeto de algoritmos.
   - **Contras:** muitas abordagens gulosas para um problema, mas apenas algumas podem funcionar; muitos problemas para os quais nenhuma abordagem gulosa é conhecida.
2. **Dividir e Conquistar**
   - **Prós:** estrutura de algoritmo clara; reduz frequentemente a complexidade (ex.: merge sort, quicksort).
   - **Contras:** a etapa de conquista pode ser muito difícil de implementar eficientemente.
3. **Programação Dinâmica**

- **Prós:** mais poderosa para problemas de otimização; transforma recursões exponenciais em soluções polinomiais ao reutilizar subsoluções.
- **Contras:** requer identificação cuidadosa de subproblemas e ordem de cálculo; pode consumir memória (tabelas) e exige prova de propriedades (subestrutura ótima e sobreposição).

Problema do Troco (Coin Change Problem)

- **INSTÃNCIA**: Seja C um conjunto de moedas $\{c_1, c_2, \cdots, c_n\}$ no qual c_i significa uma moeda de um valor específico e $c_i = c_j$ se $i = j$. Seja S o valor do troco.
- **SOLUÇÃO**: O menor número de moedas para atingir o valor S.
- **EXEMPLO**: $C = \{1, 2, 6\}$ e $S = 8$. Qual é o menor número de moedas para atingir $S = 8$? Projete um algoritmo para calcular o menor número de moedas. (Sabe-se que a resposta é 2 moedas, uma de 6 e uma de 2).

Solução 1: Enumeração do Espaço Amostral

- **Ideia:**
  Gerar **todas as combinações possíveis** de moedas que somam S e escolher a que usa o **menor número de moedas**.
- **Características:**
  - Explora **todo o espaço de busca** $\rightarrow$ considera todas as combinações.
  - **Garante a solução ótima**, mas é **computacionalmente inviável** para grandes S ou muitos tipos de moedas.
- **Complexidade:**
  $O(2^n)$, pois cada moeda pode estar ou não presente em uma combinação.

Solução 2: Algoritmo Guloso

- **Ideia:**
  Escolher **iterativamente a maior moeda possível** que **não ultrapasse o valor restante** a ser atingido.
- **Pseudocódigo:**

```
troco <- S
moedas_usadas <- 0
enquanto troco > 0:
    escolha c_max = maior moeda ≤ troco
```

```
    troco <- troco - c_max
    moedas_usadas <- moedas_usadas + 1
```

- **Complexidade:**
  $O(n)$, se as moedas estiverem ordenadas.
- **Observação:**
  O algoritmo guloso **só é ótimo** para sistemas de moedas **canônicos** () (ex.: {1, 5, 10, 25, 50}), mas **não garante a solução ótima** para qualquer conjunto arbitrário de moedas.
  - Exemplo de falha: $C = 1, 3, 4, S = 6$
    - Guloso → 4 + 1 + 1 = 3 moedas
    - Ótimo → 3 + 3 = 2 moedas

Solução 3: Programação Dinâmica

- **Ideia:**
  Utilizar uma **recorrência** para quebrar o problema em **subproblemas menores**, armazenando resultados intermediários para evitar recomputações.
- **Formulação Recursiva:**

$$M(S) = \begin{cases} 0, & \text{se } S = 0 \\ \min\limits_{c_i \leq S} \big(1 + M(S - c_i)\big), & \text{se } S > 0 \end{cases}$$

  onde $M(S)$ representa o **menor número de moedas** para atingir o valor S.
- **Complexidade:**
  $O(nS)$, polinomial, muito mais eficiente que a enumeração.

Referências

- Algorithm Design - Jon Kleinberg, Eva Tardos

FIM